DSA0101 | ASSIGNMENT 1

City Metro Smart-Card Fare Logic

Written answers for Tasks 1-3, followed by complete C++ source listings for Tasks 4, 5, 6, 7, 8, 9, 10, and 12.

Student: Lingesh .S
Register number: 1924260290

Task 1 - Problem and object analysis

Objects. A SmartCard owns an identifier and private integer-paise balance; canTravel and deduct enforce the reserve invariant. A Passenger carries a category (standard, student, senior) and eligibility evidence; concession policy belongs to fare calculation. A Journey records origin/destination zones, departure time, day type, and computed fare. Zone is a closed set of network regions. FareTable owns the mapping from zone to base fare and applies policy. Separating responsibilities prevents duplicated fare constants and balance checks.

Procedural versus OOP. Procedural code passes balance, zone, time, and category to free functions and updates balance by reference. It is compact for a tiny fixed program, but callers can bypass validation. OOP binds state and valid operations together, keeping the card invariant local. Both use O(1) time per journey and O(1) state per card. Encapsulation confines rule changes to policy classes rather than every caller.

Principles. Encapsulation: private card balance. Abstraction: charge(journey) exposes a simple operation. Inheritance: an optional ConcessionPassenger hierarchy, though a category enum or strategy is simpler for two discounts. Polymorphism: a DiscountPolicy interface can support student, senior, and later pass policies; avoid inheritance unless new policies require it.

Invariants and assumptions. Zone is valid; time is 0-23; fares are nonnegative; peak intervals are [08:00,10:00) and [17:00,20:00); surcharge is 20%, rounded to the nearest paise; concession rates need policy input because the brief does not specify percentages; a permitted trip leaves at least 5000 paise. Invalid input never mutates the card.

Value	Type	Reason
Fare, balance, total	long long paise	Exact currency; large aggregate headroom
Zone	enum class Zone	Closed, type-safe categories
Hour, minute, percent	int	Small signed quantities with validation
Peak, valid, pass result	bool	Two-state facts

Journey count	long long	Scales beyond station use

Complexity. Free functions and classes both use O(1) time per journey and O(1) extra memory per card. Procedural interface: fare(zone,time,kind) and try_deduct(balance,fare). OOP interface: FareTable::fare(journey) and SmartCard::charge(fare). Centralized policy reduces the call sites to revise when rules change.

Task 2 - Tokens, constants, and identifiers

In `constexpr long long MIN_BALANCE_PAISE = 5000;`, the tokens include keywords (constexpr, long), identifier (MIN_BALANCE_PAISE), literal constant (5000), operator (=), and punctuator (;). Use constexpr for compiletime fares and limits: it is typed, scoped, and usable in constant expressions. const suits read-only runtime values. #define performs textual substitution, lacks type and scope, and can duplicate side effects, so avoid it for fare values. enum class Zone { Inner, Middle, Outer } prevents accidental integer mixing and scopes enumerators; a plain enum converts more freely.

Identifiers begin with a letter or underscore, contain letters, digits, or underscores, and should avoid reserved underscore patterns. Use PascalCase for types, UPPER_SNAKE_CASE for fixed constants, and descriptive lower camel case for functions and variables.

A macro such as #define OUTER_FARE 3000 uses no storage, like an optimized constexpr value generally does; both have O(1) runtime impact and effectively no per-journey memory. A macro has no type checking or scope, while constexpr is checked and participates in overload and type rules. A zone enum class protects call sites. Adding a fourth zone means adding an enumerator and fare-table entry; the table size can be checked against the enum count.

Task 3 - Data types and compatibility

Displayed rupees	long long plus remainder	Avoids floating-point formatting error
Value	**Type**	**Reason**
Input token	std::string	Safe validation before conversion

For this g++ target, common sizes are bool 1, char 1, int 4, long long 8, float 4, and double 8 bytes. sizeof is implementation-defined and should be printed on the submission machine. `<climits>` provides INT_MAX (typically 2,147,483,647) and LLONG_MAX (typically 9,223,372,036,854,775,807). At 3,000,000 trips x Rs 36 = Rs 108,000,000,000 (10,800,000,000,000 paise), int overflows; long long is ample.

Integer paise avoids binary floating-point representation. Repeated 0.1 additions need not equal exact decimal 0.1*n. Both double and long long additions are O(1), and both occupy 8 bytes on the target; integer paise avoids rounding accumulation as long as overflow is prevented. Usual arithmetic conversions can promote int to double; bool promotes to int in arithmetic. Use explicit casts at intentional conversion boundaries.

-

-Wall -Wextra
-

Task04 conversions

Compile independently: `g++ -std=c++17 pedantic conversions.cpp -o program`

```
001  #include <iostream>
002  int main() {
003      int a = 7, b = 2;
004      double c = a / b;
005      double d = static_cast<double>(a) / b;
006      std::cout << c << ' ' << d << '\n';
007      char ch = 'A' + 2;
008      std::cout << ch << ' ' << ch + 1 << '\n';
009      bool flag = 5;
010      std::cout << flag << ' ' << std::boolalpha << flag << '\n';
011      unsigned int u = 1;
012      int s = -1;
013      std::cout << (s < u ? "s < u" : "s >= u") << '\n'; 014
}
```

-

-Wall -Wextra -

Task05 operators

Compile independently: `g++ -std=c++17 pedantic operators.cpp -o program`

```
001  #include <iostream>
002  int main() {
003      int x = 5, y = 3;
004      std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
005      int z = x++ + ++y;
006      std::cout << z << ' ' << x << ' ' << y << '\n';
007      std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n'; 008      std::cout <<
10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\ n'; 009      int k = 0;
010      for (int i = 0; i < 10; i += 3) k += i;
011      std::cout << k << '\n'; 012
}
```

-

-Wall -Wextra -

Task06 fare

Compile independently: `g++ -std=c++17 pedantic fare.cpp -o program`

```
001  #include <iomanip>
002  #include <iostream>
003
004  enum class Zone { Inner, Middle, Outer };
005  constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
006  constexpr int PEAK_SURCHARGE_PERCENT = 20;
007  constexpr long long MIN_BALANCE_PAISE = 5000; 008
009  long long fare(Zone z, bool peak) {
010      long long base = 0;
011      switch (z) {
012      case Zone::Inner: base = BASE_PAISE[0]; break;
013      case Zone::Middle: base = BASE_PAISE[1]; break;
014      case Zone::Outer: base = BASE_PAISE[2]; break;
015      default: return -1;
016      }
017      if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
018      return base;
019  }
020  bool canTravel(long long balance, long long f) { 021      return f >=
0 && balance >= f && balance - f >= MIN_BALANCE_PAISE; 022
}
023  int main() {
024      long long balance = 10000;
025      struct Trip { Zone zone; bool peak; }; 026      const Trip trips[] =
{{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle,  false}}; 027      for (const
Trip& t : trips) {
028          const long long f = fare(t.zone, t.peak);
029          if (canTravel(balance, f)) {
030              balance -= f;
031              std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
032                        << std::setfill('0') << f % 100 << std::setfill(' '); 033
} else {
034              std::cout << "Denied (low balance)";
035          }
036          std::cout << " balance Rs " << balance / 100 << '\n';
037      }
038  }
```

Task07 - smart_card

Compile independently: `g++ -std=c++17 -Wall -Wextra -pedantic smart_card.cpp -o program`

```
001  #include <iostream>
002  constexpr long long MIN_BALANCE_PAISE = 5000;
003  class SmartCard {
004  public:
005      explicit SmartCard(long long initial) : balance_(initial) {}
006      bool canTravel(long long fare) const {
007          return fare >= 0 && balance_ >= fare && balance_ - fare >=
MIN_BALANCE_PAISE;
008      }
009      bool deduct(long long fare) {
010          if (!canTravel(fare)) return false;
011          balance_ -= fare;
012          return true;
013      }
014      long long balance() const { return balance_; }
015  private:
```

```
-
                                                        -Wall -Wextra -
016      long long balance_; 017
};
```

```
int main() {
    SmartCard card(10000);
    for (long long fare : {1000LL, 3600LL, 2000LL}) {
        std::cout << (card.deduct(fare) ? "Accepted" : "Denied")
                  << " balance_paise=" << card.balance() << '\n';
    } 024
}
```

Task08 - input

Compile independently: `g++ -std=c++17 -Wall -Wextra -pedantic input.cpp -o program`

```
001  #include <iostream>
002  #include <limits>
003  #include <sstream>
004  #include <string>
005
006  bool readInt(const std::string& prompt, int low, int high, int& value) { 007
for (;;) {
008          std::cout << prompt;
009          std::string line;
010          if (!std::getline(std::cin, line)) return false;
011          std::istringstream in(line);
012          char extra{}; 013          if ((in >> value) && !(in >> extra) && value
>= low && value <= high) return true; 014          std::cout << "Invalid input; enter a
whole number from " << low << " to "
<< high << ".\n";
015      }
016  }
017  int main() {
018      int hour{};
019      if (!readInt("Hour (0-23): ", 0, 23, hour)) return 1;
020      const bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
021      std::cout << (peak ? "Peak" : "Off-peak") << '\n'; 022
}
```

Task09 - fare_tests

Compile independently: `g++ -std=c++17 -Wall -Wextra -pedantic fare_tests.cpp -o program`

```
001  #include <iostream>
002  #include <sstream>
003  #include <string>
004
005  enum class Zone { Inner, Middle, Outer };
006  long long fare(Zone z, int hour) {
007      long long base = z == Zone::Inner ? 1000 : z == Zone::Middle ? 2000 : 3000;
008      const bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
009      return peak ? base + base * 20 / 100 : base; 010
}
011  bool canTravel(long long bal, long long f) { return bal >= f && bal - f >= 5000; }
012  bool parseZone(const std::string& text, Zone& zone) {
013      if (text == "inner") zone = Zone::Inner;
014      else if (text == "middle") zone = Zone::Middle;
015      else if (text == "outer") zone = Zone::Outer;
016      else return false;
017      return true;
018  }
019  bool parseNonnegativeRecharge(const std::string& text, long long& amount) {
020      std::istringstream input(text);
021      char extra{};
022      return (input >> amount) && !(input >> extra) && amount >= 0; 023
}
024  bool parseHour(const std::string& text, int& hour) { 025
std::istringstream input(text);
026      char extra{};
027      return (input >> hour) && !(input >> extra) && hour >= 0 && hour <= 23; 028
}
029  struct Case { const char* name; Zone z; int hour; long long expected; };
030  int main() {
031      const Case cases[] = {
032          {"inner offpeak", Zone::Inner, 7, 1000}, {"middle offpeak", Zone::Middle,
12, 2000},
033          {"outer offpeak", Zone::Outer, 12, 3000}, {"08:00", Zone::Inner, 8, 1200},
034          {"10:00", Zone::Inner, 10, 1000}, {"17:00", Zone::Middle, 17, 2400},
035          {"20:00", Zone::Outer, 20, 3000}
036      };
037      bool all = true;
038      for (const Case& c : cases) {
039          const long long actual = fare(c.z, c.hour);
040          const bool pass = actual == c.expected;
041          all = all && pass;
042          std::cout << c.name << ": expected=" << c.expected << " actual=" << actual
043                    << (pass ? " PASS\n" : " FAIL\n"); 044
}
045      const bool exactReserve = canTravel(6000, 1000);
046      const bool belowReserve = canTravel(5999, 1000);
047      Zone parsedZone{};
048      long long recharge{};
049      int parsedHour{};
050      const bool unknownZoneRejected = !parseZone("airport", parsedZone); 051
const bool negativeRechargeRejected = !parseNonnegativeRecharge("-500", recharge);
052      const bool lettersHourRejected = !parseHour("eight", parsedHour);
053      std::cout << "exact Rs50 reserve=" << (exactReserve ? "PASS" : "FAIL") << '\n'
054                << "below reserve denied=" << (!belowReserve ? "PASS" : "FAIL") << '\
n' 055                << "negative recharge rejected=" << (!negativeRechargeRejected ?
"PASS" : "FAIL") << '\n' 056                << "letters for hour rejected=" <<
(lettersHourRejected ? "PASS" :
```

```
"FAIL") << '\n' 057                << "unknown zone rejected=" <<
(unknownZoneRejected ? "PASS" :
"FAIL") << '\n';
058      return all && exactReserve && !belowReserve && negativeRechargeRejected &&
059                     lettersHourRejected && unknownZoneRejected ? 0 : 1; 060
}
```

Task10 - totals

Compile independently: `g++ -std=c++17 -Wall -Wextra -pedantic totals.cpp -o program`

```
001  #include <iomanip>
002  #include <iostream>
003  #include <limits>
004  int main() {
005      constexpr long long farePaise = 3600;
006      for (long long n : {20000LL, 500000LL, 3000000LL}) {
007          const long long total = n * farePaise;
008          const double rupeeTotal = static_cast<double>(n) * 36.0;
009          std::cout << n << " trips: " << total << " paise = Rs " << total / 100 010
<< "; double Rs " << std::fixed << std::setprecision(2) << rupeeTotal << '\n'; 011      }
012      std::cout << "sizeof(int)=" << sizeof(int) << " sizeof(long long)=" << sizeof(
long long) 013                << " sizeof(double)=" << sizeof(double) << " INT_MAX=" <<
std::  numeric_limits<int>::max()  014                              <<  "  LLONG_MAX="  <<
std::numeric_limits<long long>::max() << '\n'; 015  }
```

Task12 - revised

Compile independently: `g++ -std=c++17 -Wall -Wextra -pedantic revised.cpp -o program`

```
001  #include <array>
002  #include <iostream>
003  #include <optional>
004
005  enum class Zone : int { Inner, Middle, Outer, Airport, Count };
006  constexpr std::array<long long, static_cast<int>(Zone::Count)> BASE_PAISE{1000,
2000, 3000, 6000};
007  constexpr int PEAK_PERCENT = 20;
008  constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
009  constexpr long long MIN_BALANCE_PAISE = 5000; 010
011  std::optional<long long> fare(Zone zone, int hour, bool weekend) {
012      const int index = static_cast<int>(zone);
013      if (index < 0 || index >= static_cast<int>(Zone::Count) || hour < 0 || hour >
23) return std::nullopt;
014      const long long base = BASE_PAISE[static_cast<std::size_t>(index)];
015      if (weekend) return base - base * WEEKEND_DISCOUNT_PERCENT / 100;
016      const bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
017      return peak ? base + base * PEAK_PERCENT / 100 : base;
018     }   019     class
MonthlyPass     {     020
public:
021      MonthlyPass(Zone first, Zone second) : first_(first), second_(second) {} 022
bool covers(Zone zone) const { return zone == first_ || zone == second_; }
023  private:
024      Zone first_, second_;
025  };
026  class SmartCard {
027  public:
028      explicit SmartCard(long long balance) : balance_(balance) {}
029      bool charge(long long amount) {
030          if (amount < 0 || balance_ < amount || balance_ - amount <
MIN_BALANCE_PAISE) return false;
031          balance_ -= amount;
032          return true;
033      }
034      long long balance() const { return balance_; }
035  private:
036      long long balance_;
037  };
038  int main() {
039      const auto airportPeak = fare(Zone::Airport, 8, false);
040      const auto airportWeekendPeak = fare(Zone::Airport, 8, true);
041      std::cout << "Airport weekday peak=" << *airportPeak << " paise (expected
7200)\n" 042                << "Airport weekend peak=" << *airportWeekendPeak <<
" paise ( expected 5400)\n"; 043      const MonthlyPass pass(Zone::Inner, Zone::Outer);
044      std::cout << "pass covers Inner=" << pass.covers(Zone::Inner)
045                << " Middle=" << pass.covers(Zone::Middle) << '\n'; 046
SmartCard card(10000); 047      std::cout << "charge Rs 20=" <<
card.charge(2000) << " balance=" << card.
balance() 048                << " charge Rs 10=" << card.charge(1000) << "
balance=" << card. balance()
<< '\n';
049      std::cout << "invalid zone=" << (fare(static_cast<Zone>(99), 12, false).
has_value() ? "accepted" : "rejected") << '\n';
050  }
```

Page

https://github.com/lingeshsivakumar77-svg/DSA0101

Page